8086 Assembly Programming

Hamim Ibne Nasim's notes

CSE341: Microprocessor

BRAC University

February 26, 2026

Contents

1 Basic Operations

1.1 Task: Input and Move Data

Take input in register AX and move it to BX.

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
    ; Input number in AX
    MOV AH, 1        ; DOS function for single character input
    INT 21H
    MOV BL, AL       ; Store first digit

    ; Move to BX
    MOV BH, 0


    MOV AX, 4C00H      ; Exit
    INT 21H
MAIN ENDP
END MAIN
```

Listing 1: Input and Move

1.2 Task: Swap Two Numbers (Using 3 Registers)

Swap values in AX and BX using temporary register CX.

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
    MOV AX, 10       ; First number
    MOV BX, 20       ; Second number

    ; Swap using CX as temporary
    MOV CX, AX       ; CX = AX
    MOV AX, BX       ; AX = BX
    MOV BX, CX       ; BX = CX

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 2: Swap with MOV

1.3 Task: Swap Using ADD/SUB Only

Swap two numbers without using a third register.

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
    MOV AX, 5        ; First number
    MOV BX, 10       ; Second number

    ; Swap algorithm
    ADD AX, BX       ; AX = AX + BX
    SUB BX, AX       ; BX = BX - AX (now BX = original BX - (
       original AX + original BX) = -original AX)
    NEG BX           ; BX = original AX
    SUB AX, BX       ; AX = (original AX + original BX) - original
        AX = original BX

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 3: Swap with ADD/SUB

1.4 Task: Add Two Numbers

```
.MODEL SMALL
.STACK 100H
.DATA
    NUM1 DB 5
    NUM2 DB 3
    RESULT DB ?
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AL, NUM1
    ADD AL, NUM2     ; AL = AL + NUM2
    MOV RESULT, AL

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 4: Addition

1.5 Task: Subtract Two Numbers

Note: Subtracting larger from smaller gives negative result (2's complement).

```
.MODEL SMALL
.STACK 100H
.DATA
    NUM1 DB 10
    NUM2 DB 7
    RESULT DB ?
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AL, NUM1
    SUB AL, NUM2     ; AL = AL - NUM2
    MOV RESULT, AL

    ; If NUM2 > NUM1, result is in 2's complement (negative)

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 5: Subtraction

1.6 Task: Arithmetic Operations with Variables

```
.MODEL SMALL
.STACK 100H
.DATA
    A DW 10
    B DW 5
    C DW ?
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; 1. A = B - A
    MOV AX, B
    SUB AX, A
    MOV A, AX

    ; 2. A = -(A + 1)
    MOV AX, A
    ADD AX, 1
    NEG AX
    MOV A, AX
```

```
22
23      ; 3. C = A + (B + 1)
24      MOV AX, B
25      INC AX              ; AX = B + 1
26      ADD AX, A
27      MOV C, AX
28
29      ; 4. A = B - (A - 1)
30      MOV AX, A
31      DEC AX              ; AX = A - 1
32      MOV BX, B
33      SUB BX, AX
34      MOV A, BX
35
36      MOV AH, 4CH
37      INT 21H
38  MAIN ENDP
39  END MAIN
```

Listing 6: Variable Arithmetic

1.7 Task: Multiplication and Division

```
1   .MODEL SMALL
2   .STACK 100H
3   .DATA
4       X DW 6
5       Y DW 3
6       Z DW 2
7       RESULT DW ?
8   .CODE
9   MAIN PROC
10      MOV AX, @DATA
11      MOV DS, AX
12
13      ; 1. X * Y
14      MOV AX, X
15      MUL Y               ; DX:AX = AX * Y
16      MOV RESULT, AX
17
18      ; 2. X / Y
19      MOV AX, X
20      MOV DX, 0           ; Clear DX for division
21      DIV Y               ; AX = DX:AX / Y, DX = remainder
22      MOV RESULT, AX
23
24      ; 3. X * Y / Z
25      MOV AX, X
26      MUL Y               ; DX:AX = X * Y
27      DIV Z               ; AX = (X*Y) / Z
28      MOV RESULT, AX
```

```
29
30      MOV AH, 4CH
31      INT 21H
32  MAIN ENDP
33  END MAIN
```

Listing 7: MUL and DIV Operations

1.8 Task: Complex Arithmetic Expression

Calculate: (1 + 2) * (3 - 1) / 5 + 3 + 2 - (1 * 2)

```
1   .MODEL SMALL
2   .STACK 100H
3   .DATA
4       RESULT DW ?
5   .CODE
6   MAIN PROC
7       ; (1 + 2) = 3
8       MOV AX, 1
9       ADD AX, 2           ; AX = 3
10      MOV BX, AX          ; BX = 3
11
12      ; (3 - 1) = 2
13      MOV AX, 3
14      SUB AX, 1           ; AX = 2
15
16      ; 3 * 2 = 6
17      MUL BX              ; AX = 3 * 2 = 6
18
19      ; 6 / 5 = 1
20      MOV BX, 5
21      MOV DX, 0
22      DIV BX              ; AX = 6 / 5 = 1
23
24      ; + 3
25      ADD AX, 3           ; AX = 4
26
27      ; + 2
28      ADD AX, 2           ; AX = 6
29
30      ; (1 * 2) = 2
31      MOV BX, 1
32      MOV CX, 2
33      PUSH AX             ; Save current result
34      MOV AX, BX
35      MUL CX              ; AX = 2
36      MOV BX, AX
37      POP AX              ; Restore result
38
39      ; - 2
40      SUB AX, BX          ; AX = 6 - 2 = 4
```

```
41
42      MOV RESULT, AX
43
44      MOV AH, 4CH
45      INT 21H
46  MAIN ENDP
47  END MAIN
```

Listing 8: Complex Expression

2 Input/Output Operations

2.1 Task: Character Input and Display

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG DB 'Please insert a character: $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Display message
    LEA DX, MSG
    MOV AH, 9
    INT 21H

    ; Input character
    MOV AH, 1
    INT 21H
    MOV BL, AL      ; Save character

    ; Display character
    MOV DL, BL
    MOV AH, 2
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 9: Character I/O

2.2 Task: Addition with User Input

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG1 DB 'Enter first number: $'
    MSG2 DB 0DH, 0AH, 'Enter second number: $'
    MSG3 DB 0DH, 0AH, 'Sum is: $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Prompt for first number
    LEA DX, MSG1
```

```
    MOV AH, 9
    INT 21H

    ; Input first digit
    MOV AH, 1
    INT 21H
    SUB AL, 30H      ; Convert ASCII to number
    MOV BL, AL

    ; Prompt for second number
    LEA DX, MSG2
    MOV AH, 9
    INT 21H

    ; Input second digit
    MOV AH, 1
    INT 21H
    SUB AL, 30H

    ; Add numbers
    ADD BL, AL

    ; Display result message
    LEA DX, MSG3
    MOV AH, 9
    INT 21H

    ; Display sum
    MOV DL, BL
    ADD DL, 30H      ; Convert to ASCII
    MOV AH, 2
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 10: Interactive Addition

2.3 Task: Uppercase to Lowercase Conversion

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG DB 'Enter an uppercase letter: $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

```

```
    ; Display prompt
    LEA DX, MSG
    MOV AH, 9
    INT 21H

    ; Input uppercase letter
    MOV AH, 1
    INT 21H

    ; Convert to lowercase (add 32 or 20H)
    ADD AL, 20H

    ; Move to next line
    MOV DL, 0DH
    MOV AH, 2
    INT 21H
    MOV DL, 0AH
    INT 21H

    ; Display lowercase
    MOV DL, AL
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 11: Case Conversion

2.4 Task: Sum of Two Digits with Message

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG1 DB 0DH, 0AH, 'THE SUM OF $'
    MSG2 DB ' AND $'
    MSG3 DB ' IS $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Display ?
    MOV DL, '?'
    MOV AH, 2
    INT 21H

    ; Input first digit
    MOV AH, 1
    INT 21H
```

```
    MOV BL, AL      ; Save first digit
    SUB AL, 30H
    MOV CL, AL      ; CL = first number

    ; Input second digit
    INT 21H
    MOV BH, AL      ; Save second digit
    SUB AL, 30H

    ; Calculate sum
    ADD CL, AL      ; CL = sum

    ; Display message
    LEA DX, MSG1
    MOV AH, 9
    INT 21H

    ; Display first digit
    MOV DL, BL
    MOV AH, 2
    INT 21H

    ; Display "AND"
    LEA DX, MSG2
    MOV AH, 9
    INT 21H

    ; Display second digit
    MOV DL, BH
    MOV AH, 2
    INT 21H

    ; Display "IS"
    LEA DX, MSG3
    MOV AH, 9
    INT 21H

    ; Display sum
    MOV DL, CL
    ADD DL, 30H
    MOV AH, 2
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 12: Digit Sum Display

2.5 Task: Display Initials Vertically

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG DB 'ENTER THREE INITIALS: $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Display prompt
    LEA DX, MSG
    MOV AH, 9
    INT 21H

    ; Input three characters
    MOV AH, 1
    INT 21H
    MOV BL, AL      ; First initial

    INT 21H
    MOV BH, AL      ; Second initial

    INT 21H
    MOV CL, AL      ; Third initial

    ; New line
    MOV DL, 0DH
    MOV AH, 2
    INT 21H
    MOV DL, 0AH
    INT 21H

    ; Display first initial
    MOV DL, BL
    INT 21H

    ; New line
    MOV DL, 0DH
    INT 21H
    MOV DL, 0AH
    INT 21H

    ; Display second initial
    MOV DL, BH
    MOV AH, 2
    INT 21H

    ; New line
    MOV DL, 0DH
```

```
    INT 21H
    MOV DL, 0AH
    INT 21H

    ; Display third initial
    MOV DL, CL
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 13: Vertical Display

2.6 Task: Hex to Decimal Conversion

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG1 DB 'ENTER A HEX DIGIT: $'
    MSG2 DB 0DH, 0AH, 'IN DECIMAL IT IS $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Display prompt
    LEA DX, MSG1
    MOV AH, 9
    INT 21H

    ; Input hex digit (A-F)
    MOV AH, 1
    INT 21H

    ; Convert to decimal
    SUB AL, 37H      ; 'A' (65) - 37H = 10
    MOV BL, AL

    ; Display message
    LEA DX, MSG2
    MOV AH, 9
    INT 21H

    ; Display tens digit (1)
    MOV DL, '1'
    MOV AH, 2
    INT 21H

    ; Display units digit
```

```
    MOV DL, BL
    ADD DL, 30H
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 14: Hex to Decimal

2.7 Task: Display 10x10 Box of Asterisks

```
.MODEL SMALL
.STACK 100H
.DATA
    LINE DB '**********', 0DH, 0AH, '$'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV CX, 10      ; 10 rows

PRINT_ROW:
    LEA DX, LINE
    MOV AH, 9
    INT 21H

    LOOP PRINT_ROW

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 15: Asterisk Box

3 Conditional Statements

3.1 Task: Replace Negative with 5

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
    MOV AX, -10      ; Test with negative number

    CMP AX, 0
    JGE POSITIVE     ; Jump if AX >= 0

    ; AX is negative
    MOV AX, 5

POSITIVE:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 16: Negative Number Check

3.2 Task: Display Character that Comes First

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
    MOV AL, 'B'      ; First character
    MOV BL, 'A'      ; Second character

    CMP AL, BL
    JL DISPLAY_AL    ; If AL < BL

    ; Display BL
    MOV DL, BL
    MOV AH, 2
    INT 21H
    JMP EXIT

DISPLAY_AL:
    MOV DL, AL
    MOV AH, 2
    INT 21H

EXIT:
```

```
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 17: Character Comparison

3.3 Task: Sign Indicator

Put -1 in BX if AX is negative, 0 if zero, 1 if positive.

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
    MOV AX, 10        ; Test value

    CMP AX, 0
    JL NEGATIVE
    JE ZERO

    ; Positive
    MOV BX, 1
    JMP EXIT

NEGATIVE:
    MOV BX, -1
    JMP EXIT

ZERO:
    MOV BX, 0

EXIT:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 18: Sign Detection

3.4 Task: Odd/Even Display

If AL contains 1 or 3, display "o"; if 2 or 4, display "e".

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
    MOV AL, 3         ; Test value

```

```
    CMP AL, 1
    JE ODD
    CMP AL, 3
    JE ODD
    CMP AL, 2
    JE EVEN
    CMP AL, 4
    JE EVEN
    JMP EXIT

ODD:
    MOV DL, 'o'
    MOV AH, 2
    INT 21H
    JMP EXIT

EVEN:
    MOV DL, 'e'
    MOV AH, 2
    INT 21H

EXIT:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 19: Odd/Even Character

3.5 Task: Display Uppercase Letter

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG DB 'Enter a character: $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Display prompt
    LEA DX, MSG
    MOV AH, 9
    INT 21H

    ; Input character
    MOV AH, 1
    INT 21H

    ; Check if uppercase (A-Z: 65-90)
    CMP AL, 'A'
```

```
    JL EXIT
    CMP AL, 'Z'
    JG EXIT

    ; Display uppercase letter
    MOV DL, AL
    MOV AH, 2
    INT 21H

EXIT:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 20: Uppercase Filter

3.6 Task: Even or Odd Number Check

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG_EVEN DB 'EVEN$'
    MSG_ODD DB 'ODD$'
    NUM DW 10
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AX, NUM
    MOV BL, 2
    DIV BL          ; Divide by 2

    CMP AH, 0       ; Check remainder
    JE EVEN

    ; Odd
    LEA DX, MSG_ODD
    MOV AH, 9
    INT 21H
    JMP EXIT

EVEN:
    LEA DX, MSG_EVEN
    MOV AH, 9
    INT 21H

EXIT:
    MOV AH, 4CH
    INT 21H
```

```
MAIN ENDP
END MAIN
```

Listing 21: Even/Odd Check

3.7 Task: Vowel or Consonant

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG1 DB 'Enter a letter: $'
    MSG2 DB 0DH, 0AH, 'VOWEL$'
    MSG3 DB 0DH, 0AH, 'CONSONANT$'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Display prompt
    LEA DX, MSG1
    MOV AH, 9
    INT 21H

    ; Input character
    MOV AH, 1
    INT 21H

    ; Check vowels: A, E, I, O, U (case-insensitive)
    CMP AL, 'A'
    JE VOWEL
    CMP AL, 'a'
    JE VOWEL
    CMP AL, 'E'
    JE VOWEL
    CMP AL, 'e'
    JE VOWEL
    CMP AL, 'I'
    JE VOWEL
    CMP AL, 'i'
    JE VOWEL
    CMP AL, 'O'
    JE VOWEL
    CMP AL, 'o'
    JE VOWEL
    CMP AL, 'U'
    JE VOWEL
    CMP AL, 'u'
    JE VOWEL

    ; Consonant
    LEA DX, MSG3
```

```
    MOV AH, 9
    INT 21H
    JMP EXIT

VOWEL:
    LEA DX, MSG2
    MOV AH, 9
    INT 21H

EXIT:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 22: Vowel/Consonant Check

3.8 Task: Divisibility by 5 and 11

```
.MODEL SMALL
.STACK 100H
.DATA
    MSG1 DB 'Divisible by both 5 and 11$'
    MSG2 DB 'Not divisible$'
    NUM DW 55
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Check divisibility by 5
    MOV AX, NUM
    MOV BL, 5
    DIV BL
    CMP AH, 0
    JNE NOT_DIV

    ; Check divisibility by 11
    MOV AX, NUM
    MOV BL, 11
    DIV BL
    CMP AH, 0
    JNE NOT_DIV

    ; Divisible by both
    LEA DX, MSG1
    MOV AH, 9
    INT 21H
    JMP EXIT

NOT_DIV:
```

```
    LEA DX, MSG2
    MOV AH, 9
    INT 21H

EXIT:
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 23: Divisibility Check

3.9 Task: Find Maximum of Three Numbers

```
.MODEL SMALL
.STACK 100H
.DATA
    NUM1 DW 2
    NUM2 DW 3
    NUM3 DW 4
    MAX_VAL DW ?
    MSG DB 'Maximum number is: $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AX, NUM1
    MOV BX, NUM2

    ; Compare NUM1 and NUM2
    CMP AX, BX
    JG CHECK_THIRD
    MOV AX, BX      ; BX is larger

CHECK_THIRD:
    ; Compare with NUM3
    MOV BX, NUM3
    CMP AX, BX
    JG FOUND_MAX
    MOV AX, BX

FOUND_MAX:
    MOV MAX_VAL, AX

    ; Display message
    LEA DX, MSG
    MOV AH, 9
    INT 21H

    ; Display maximum (assuming single digit)
```

```
    MOV DL, BYTE PTR MAX_VAL
    ADD DL, 30H
    MOV AH, 2
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 24: Maximum of Three

3.10 Task: Day of Week Display

```
.MODEL SMALL
.STACK 100H
.DATA
    DAY DB 3        ; 1=Saturday, 2=Sunday, etc.
    SAT DB 'Saturday$'
    SUN DB 'Sunday$'
    MON DB 'Monday$'
    TUE DB 'Tuesday$'
    WED DB 'Wednesday$'
    THU DB 'Thursday$'
    FRI DB 'Friday$'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AL, DAY

    CMP AL, 1
    JE SATURDAY
    CMP AL, 2
    JE SUNDAY
    CMP AL, 3
    JE MONDAY
    CMP AL, 4
    JE TUESDAY
    CMP AL, 5
    JE WEDNESDAY
    CMP AL, 6
    JE THURSDAY
    CMP AL, 7
    JE FRIDAY
    JMP EXIT

SATURDAY:
    LEA DX, SAT
    JMP DISPLAY
```

```
38
39 SUNDAY:
40     LEA DX, SUN
41     JMP DISPLAY
42
43 MONDAY:
44     LEA DX, MON
45     JMP DISPLAY
46
47 TUESDAY:
48     LEA DX, TUE
49     JMP DISPLAY
50
51 WEDNESDAY:
52     LEA DX, WED
53     JMP DISPLAY
54
55 THURSDAY:
56     LEA DX, THU
57     JMP DISPLAY
58
59 FRIDAY:
60     LEA DX, FRI
61
62 DISPLAY:
63     MOV AH, 9
64     INT 21H
65
66 EXIT:
67     MOV AH, 4CH
68     INT 21H
69 MAIN ENDP
70 END MAIN
```

Listing 25: Day Name Display

4 Loop Operations

4.1 Task: Display 80 Stars

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
    MOV CX, 80      ; Loop counter

PRINT_STAR:
    MOV DL, '*'
    MOV AH, 2
    INT 21H

    LOOP PRINT_STAR

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 26: Simple Loop

4.2 Task: Sum of Arithmetic Series

Calculate sum: $1 + 4 + 7 + ... + 148$

```
.MODEL SMALL
.STACK 100H
.DATA
    SUM DW 0
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AX, 0       ; Sum accumulator
    MOV BX, 1       ; Current number

SUM_LOOP:
    ADD AX, BX      ; Add to sum
    ADD BX, 3       ; Next number (increment by 3)

    CMP BX, 148
    JLE SUM_LOOP    ; Continue if BX <= 148

    MOV SUM, AX

    MOV AH, 4CH
```

```
    INT 21H
MAIN ENDP
END MAIN
```

Listing 27: Arithmetic Series Sum

4.3 Task: Multiplication by Repeated Addition

```
.MODEL SMALL
.STACK 100H
.DATA
    M DW 5
    N DW 4
    PRODUCT DW 0
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AX, 0        ; Product = 0
    MOV BX, M
    MOV CX, N

MULT_LOOP:
    ADD AX, BX       ; Product = Product + M
    LOOP MULT_LOOP   ; Decrement CX and loop

    MOV PRODUCT, AX

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 28: Multiplication Loop

4.4 Task: Display Extended ASCII Characters

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
    MOV CX, 128      ; 128 characters (80h to FFh)
    MOV DL, 80H      ; Start from 80h
    MOV BL, 0        ; Counter for 10 per line

DISPLAY_CHAR:
    MOV AH, 2
    INT 21H
```

```
13
14      ; Space after character
15      PUSH DX
16      MOV DL, ' '
17      INT 21H
18      POP DX
19
20      INC DL
21      INC BL
22
23      ; Check if 10 characters printed
24      CMP BL, 10
25      JNE CONTINUE
26
27      ; New line
28      PUSH DX
29      MOV DL, 0DH
30      INT 21H
31      MOV DL, 0AH
32      INT 21H
33      POP DX
34
35      MOV BL, 0        ; Reset counter
36
37  CONTINUE:
38      LOOP DISPLAY_CHAR
39
40      MOV AH, 4CH
41      INT 21H
42  MAIN ENDP
43  END MAIN
```

Listing 29: Extended ASCII Display

4.5 Task: Hex to Decimal Converter (Loop)

```
1   .MODEL SMALL
2   .STACK 100H
3   .DATA
4       MSG1 DB 'ENTER A HEX DIGIT: $'
5       MSG2 DB 0DH, 0AH, 'IN DECIMAL IT IS $'
6       MSG3 DB 0DH, 0AH, 'DO YOU WANT TO DO IT AGAIN? $'
7   .CODE
8   MAIN PROC
9       MOV AX, @DATA
10      MOV DS, AX
11
12  START:
13      ; Display prompt
14      LEA DX, MSG1
15      MOV AH, 9
```

```
    INT 21H

    ; Input hex digit
    MOV AH, 1
    INT 21H
    MOV BL, AL

    ; Check if valid (0-9 or A-F)
    CMP BL, '0'
    JL INVALID
    CMP BL, '9'
    JLE VALID_DIGIT
    CMP BL, 'A'
    JL INVALID
    CMP BL, 'F'
    JG INVALID
    JMP VALID_HEX

INVALID:
    ; Error handling (skip for brevity)
    JMP START

VALID_DIGIT:
    SUB BL, 30H      ; Convert 0-9
    JMP DISPLAY

VALID_HEX:
    SUB BL, 37H      ; Convert A-F (A=10, etc.)

DISPLAY:
    ; Display decimal message
    LEA DX, MSG2
    MOV AH, 9
    INT 21H

    ; Display tens digit if >= 10
    CMP BL, 10
    JL SINGLE_DIGIT

    MOV DL, '1'
    MOV AH, 2
    INT 21H
    SUB BL, 10

SINGLE_DIGIT:
    MOV DL, BL
    ADD DL, 30H
    MOV AH, 2
    INT 21H

    ; Ask to continue
```

```
    LEA DX, MSG3
    MOV AH, 9
    INT 21H

    MOV AH, 1
    INT 21H

    CMP AL, 'y'
    JE START
    CMP AL, 'Y'
    JE START

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 30: Hex Converter with Loop

4.6 Task: Sum Numbers Divisible by Last Digit of ID

```
.MODEL SMALL
.STACK 100H
.DATA
    LAST_DIGIT DB 6      ; Last non-zero digit of ID
    M DW 0               ; Sum of divisible numbers
    N DW 0               ; Sum of non-divisible numbers
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV CX, 100          ; Loop from 1 to 100
    MOV BX, 1            ; Current number

LOOP_START:
    MOV AX, BX
    MOV DL, LAST_DIGIT
    DIV DL               ; Divide by last digit

    CMP AH, 0            ; Check remainder
    JE DIVISIBLE

    ; Not divisible
    ADD N, BX
    JMP NEXT

DIVISIBLE:
    ADD M, BX

NEXT:
```

```
    INC BX
    LOOP LOOP_START

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 31: Conditional Sum Loop

4.7 Task: Number Pattern (Nested Loop)

```
.MODEL SMALL
.STACK 100H
.DATA
    INPUT DB 5          ; User input
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV CH, 0
    MOV CL, INPUT       ; Outer loop counter
    MOV BL, 1           ; Current row

OUTER_LOOP:
    PUSH CX             ; Save outer counter
    MOV CL, BL          ; Inner loop counter
    MOV DL, '1'         ; Start from 1

INNER_LOOP:
    MOV AH, 2
    INT 21H

    ; Space after number
    PUSH DX
    MOV DL, ' '
    INT 21H
    POP DX

    INC DL
    DEC CL
    JNZ INNER_LOOP

    ; New line
    PUSH DX
    MOV DL, 0DH
    MOV AH, 2
    INT 21H
    MOV DL, 0AH
    INT 21H
```

```
    POP DX

    INC BL
    POP CX                ; Restore outer counter
    LOOP OUTER_LOOP

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 32: Pattern Display

4.8 Task: Leap Year Checker

```
.MODEL SMALL
.STACK 100H
.DATA
    YEAR DW 1996
    MSG_LEAP DB 'Leap year$'
    MSG_NOT DB 'Not leap year$'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Check divisibility by 400
    MOV AX, YEAR
    MOV BX, 400
    MOV DX, 0
    DIV BX
    CMP DX, 0
    JE LEAP

    ; Check divisibility by 100
    MOV AX, YEAR
    MOV BX, 100
    MOV DX, 0
    DIV BX
    CMP DX, 0
    JE NOT_LEAP

    ; Check divisibility by 4
    MOV AX, YEAR
    MOV BX, 4
    MOV DX, 0
    DIV BX
    CMP DX, 0
    JE LEAP

NOT_LEAP:
```

```
37      LEA DX, MSG_NOT
38      JMP DISPLAY
39
40  LEAP:
41      LEA DX, MSG_LEAP
42
43  DISPLAY:
44      MOV AH, 9
45      INT 21H
46
47      MOV AH, 4CH
48      INT 21H
49  MAIN ENDP
50  END MAIN
```

Listing 33: Leap Year Detection

5 String Operations

5.1 Task: Palindrome Checker

```
.MODEL SMALL
.STACK 100H
.DATA
    TEXT DB 'race car$'
    CLEAN DB 30 DUP('$')
    IS_PALINDROME DB 0
    MSG1 DB 'Palindrome$'
    MSG2 DB 'Not Palindrome$'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    ; Remove spaces - Loop 1
    LEA SI, TEXT
    LEA DI, CLEAN
    MOV CX, 0           ; Clean length counter

REMOVE_SPACES:
    MOV AL, [SI]
    CMP AL, '$'
    JE CHECK_PALINDROME

    CMP AL, ' '         ; Skip space
    JE SKIP

    MOV [DI], AL
    INC DI
    INC CX

SKIP:
    INC SI
    JMP REMOVE_SPACES

CHECK_PALINDROME:
    ; Check if palindrome - Loop 2
    LEA SI, CLEAN       ; Left pointer
    LEA DI, CLEAN
    ADD DI, CX
    DEC DI              ; Right pointer

    MOV BX, CX
    SHR BX, 1           ; BX = length / 2

COMPARE_LOOP:
    MOV AL, [SI]
    MOV AH, [DI]
```

```
48      CMP AL, AH
49      JNE NOT_PALINDROME
50
51      INC SI
52      DEC DI
53      DEC BX
54      JNZ COMPARE_LOOP
55
56      ; Is palindrome
57      MOV IS_PALINDROME, 1
58      LEA DX, MSG1
59      JMP DISPLAY
60
61  NOT_PALINDROME:
62      MOV IS_PALINDROME, 0
63      LEA DX, MSG2
64
65  DISPLAY:
66      MOV AH, 9
67      INT 21H
68
69      MOV AH, 4CH
70      INT 21H
71  MAIN ENDP
72  END MAIN
```

Listing 34: Palindrome Detection

6 Flag Operations

6.1 Task: Flag Status After Addition

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
    ; ADD AL, BL where AL=80h, BL=80h
    MOV AL, 80H
    MOV BL, 80H

    ADD AL, BL

    ; Result: AL = 00H
    ; CF = 1 (carry out)
    ; OF = 1 (signed overflow: 128+128=-256 in signed)
    ; ZF = 1 (result is zero)
    ; SF = 0 (result is positive in signed)
    ; PF = 1 (even parity)
    ; AF = 1 (auxiliary carry)

    ; To check flags, use PUSHF and analyze
    PUSHF
    POP AX
    ; AX now contains FLAGS register

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Listing 35: Flag Analysis

6.2 Explanation: Overflow in Addition

When adding two positive numbers (AX, BX both positive):

- Carry into MSB but no carry out = Signed overflow occurs
- Result appears negative (MSB=1) but both operands were positive

When adding two negative numbers (AX, BX both negative):

- Carry out of MSB but no carry in = Signed overflow occurs
- Result appears positive (MSB=0) but both operands were negative

7 Important Notes

7.1 Register Usage

- **AX, BX, CX, DX**: General purpose 16-bit registers
- **AL, BL, CL, DL**: Low byte of corresponding registers
- **AH, BH, CH, DH**: High byte of corresponding registers
- **SI, DI**: Source and destination index registers
- **SP, BP**: Stack and base pointers

7.2 Common DOS Interrupts (INT 21H)

- **AH=01H**: Single character input (result in AL)
- **AH=02H**: Single character output (character in DL)
- **AH=09H**: String output (address in DX, string ends with '$')
- **AH=4CH**: Terminate program

7.3 ASCII Conversion

- **Number to ASCII**: ADD AL, 30H (or ADD AL, '0')
- **ASCII to Number**: SUB AL, 30H (or SUB AL, '0')
- **Uppercase to Lowercase**: ADD AL, 20H
- **Lowercase to Uppercase**: SUB AL, 20H

7.4 Loop Instructions

- **LOOP label**: Decrement CX and jump if CX $\neq 0$
- **LOOPE/LOOPZ**: Loop while equal/zero
- **LOOPNE/LOOPNZ**: Loop while not equal/not zero

7.5 Jump Instructions

- **Unconditional**: JMP
- **Equal/Zero**: JE, JZ
- **Not Equal/Not Zero**: JNE, JNZ
- **Greater**: JG (signed), JA (unsigned)
- **Less**: JL (signed), JB (unsigned)
- **Greater or Equal**: JGE (signed), JAE (unsigned)
- **Less or Equal**: JLE (signed), JBE (unsigned)

7.6 Data Definition

- **DB**: Define Byte (8 bits)
- **DW**: Define Word (16 bits)
- **DD**: Define Double Word (32 bits)
- **DUP**: Duplicate values

7.7 String Terminators

- **'$'**: DOS string terminator (for INT 21H, AH=09H)
- **0 or NULL**: C-style string terminator
- **0DH, 0AH**: Carriage return and line feed (new line)